Scientific Programming in Python

13. Session: Away from Jupyter Notebooks

Ulf Krumnack

Away from Jupyter Notebooks

- standalone Python scripts
- running scripts from the command line
- writing modules
- crafting Python packages

Away from Jupyter Notebooks

- write standalone scripts
- write python modules
- Python logging
- create a package
- unit tests
- use of environment variables
-
- use .Rmd for version control

-> What goes into Git? -> Add a console script entry point

-> easier collaboration and ?grading?

-> Notebooks in Git: pragmatic approach -> optionally use tools like nbstripout (concept) ?

```
tree . . ├── data ├── pyproject.toml ├── scripts ├── src │
└── hello_science │   ├── cli.py │   ├── core.py │   └──
io.py └── tests └── test_core.py
```

5 directories, 5 files

python -m hello_science.cli –help

Session goals

- Move from exploratory notebooks to reproducible scripts and packages
- Learn a minimal, realistic project structure
- Build a small “hello” project and grow it step-by-step:
  - from `hello.py` → CLI tool → package → tests → quality tooling

Running example: Hello, science!

We will evolve this over the lecture:

- Start: print a greeting
- Add: CLI arguments, logging, file I/O, config
- Package it: `pip install -e .`, run as `hello`
- Add: tests + formatting/linting

1) Why/when to leave notebooks

Notebooks vs scripts: typical roles

- Notebooks:
  - exploration, visualization, teaching, “scratchpad”
- Scripts/packages:
  - repeatable runs, automation, reuse, deployment, collaboration

Notes:

- Aim is not “no notebooks”, but “notebooks + scripts”.

Reproducibility checklist

- Same results tomorrow?
  - fixed random seeds
  - explicit dependencies
  - deterministic preprocessing order
- Same results for others?
  - environment + entry points + documented usage

Diff-ability & review

- Notebooks:
  - JSON, outputs, execution order issues → noisy diffs
- Scripts:
  - clean diffs, easier code review, modular functions

Automation and scale

- Scripts integrate into:
  - batch runs (HPC/cluster)
  - CI pipelines
  - scheduled jobs
- No GUI required; runs headless

Rule of thumb

- Keep notebooks for:
  - exploration, prototyping, communicating results
- Move to scripts/packages when you need:
  - repeatable execution
  - parameters
  - reuse
  - testing

2) Basic script structure

Minimal standalone script

```
# hello.py
print("Hello, world!")
```

Run:

```
python hello.py
```

The Python shebang (#!)

On Linux/macOS, the *shebang* tells the shell which interpreter should run the file.

```
#!/usr/bin/env python3
print("Hello, world!")
```

Make it executable and run it like a program:

```
chmod +x hello.py
./hello.py
```

The Python shebang (`#!`)

Notes:

- `#!/usr/bin/env python3` finds `python3` from your current environment/`PATH`
- Works best inside an activated virtual environment
- On Windows this is less common; typically use `python hello.py`

Add a `main()` function

```
# hello.py
def main():
    print("Hello, world!")

if __name__ == "__main__":
    main()
```

Benefits:

- importable without side effects
- testable functions

Separate concerns: compute vs I/O

Bad pattern (hard to test):

- data reading, processing, plotting all mixed

Better:

- functions do computation
- `main()` handles arguments and I/O

Determinism: seed your randomness

```python
import numpy as np

def simulate(n, seed=0):
    rng = np.random.default_rng(seed)
    return rng.normal(size=n)
```

Common pitfalls

- Code at top-level runs on import
- Hidden state in global variables
- Paths relative to “current working directory” (CWD)

3) Command-line interfaces (CLI)

Why CLI?

- Replace “run cells in order” with:
  - `python hello.py --name Ada`
- Reproducible commands can be copied into papers, README, CI

argparse: minimal example

```python
# hello.py
import argparse

def parse_args():
    p = argparse.ArgumentParser()
    p.add_argument("--name", default="world")
    return p.parse_args()

def main():
    args = parse_args()
    print(f"Hello, {args.name}!")

if __name__ == "__main__":
    main()
```

Add options that matter in science

- Input/output paths
- Random seed
- Parameters of an algorithm (e.g., `--n-estimators`)

```
python hello.py --name "Ada"
python hello.py --name "Ada" --seed 123
```

Exit codes and errors

- Good CLI tools:
  - return non-zero on failure
  - write diagnostics to stderr (or via logging)

```
import sys
if some_error:
    print("error: ...", file=sys.stderr)
    raise SystemExit(2)
```

Help text is documentation

```
python hello.py --help
```

- good `--help` reduces support/questions dramatically

```
usage: hello-science [-h] [--name NAME] [--out PATH] [-v]

Greet from the command line.

options:
  -h, --help      show this help message and exit
  --name NAME     Name to greet (default: world)
  --out PATH      Write greeting to PATH (default: stdout)
  -v, --verbose   More logs (-v=INFO, -vv=DEBUG)
```

Help text is documentation

```
import argparse
from pathlib import Path

def parse_args(argv=None):
    p = argparse.ArgumentParser(prog="hello-science", description="Greet fr
    p.add_argument("--name", default="world", metavar="NAME", help="Name to
    p.add_argument("--out", type=Path, metavar="PATH", help="Write greeting
    p.add_argument("-v", "--verbose", action="count", default=0, help="More
    return p.parse_args(argv)
```

4) Project layout

A typical scientific repo layout

```
hello-science/
  pyproject.toml
  README.md
  src/
    hello_science/
      __init__.py
      cli.py
      core.py
  tests/
  notebooks/
  data/        # usually not committed (or only small samples)
  scripts/
```

Why `src/` layout?

- Prevents accidental imports from the working directory
- Forces you to install your package (even editable)
- Encourages clean module boundaries

Paths: avoid CWD surprises

Prefer:

```
from pathlib import Path

DATA_DIR = Path(__file__).resolve().parents[2] / "data"
```

Or pass paths via CLI/config rather than hard-coding.

Keep “library code” separate from “scripts”

- `src/hello_science/core.py`: pure functions
- `src/hello_science/cli.py`: parsing args, calling core
- `scripts/`: small utilities that import package code

What goes into Git?

- Commit:
  - source, tests, configs, small example data
- Don't commit:
  - huge data, generated outputs (`.png`, `.csv`), environments

5) Packaging basics

What is a Python package?

- A directory of modules intended for import
- Usually:
  - `hello_science/__init__.py`
  - `core.py`, `cli.py`, etc.

Minimal package code

```python
# src/hello_science/core.py
def greet(name: str) -> str:
    return f"Hello, {name}!"
```

```python
# src/hello_science/__init__.py
from .core import greet
__all__ = ["greet"]
```

pyproject.toml (minimal)

```toml
[project]
name = "hello-science"
version = "0.1.0"
dependencies = []

[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[tool.setuptools.packages.find]
where = ["src"]
```

Editable installs for development

```
python -m venv .venv
source .venv/bin/activate
pip install -e .
```

Now imports work anywhere:

```
python -c "import hello_science; print(hello_science.greet('Ada'))"
```

Public API and imports

- Decide what users import:
  - `import hello_science` vs `from hello_science.core import greet`
- Keep internal helpers private (`_helper()`)

6) Environment management

The problem

- “It works on my machine”
- Different package versions → different results

Goal:

- explicit environments + dependency pinning

venv + pip workflow

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -U pip
pip install numpy pandas matplotlib scikit-learn
```

Recording dependencies

Options:

- `requirements.txt` (simple)
- `pyproject.toml` dependencies (modern packaging)
- lockfiles (tool-specific)

Example `requirements.txt`:

```
numpy==2.1.0
pandas==2.2.3
scikit-learn==1.5.2
```

Recreating the environment

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Advice for practice

- One project → one environment
- Pin versions for assignments/research artifacts
- Document Python version too (`python --version`)

7) Running code: `python`, `-m`, entry points

Three common ways to run

1. `python hello.py`
2. `python -m hello_science.cli`
3. installed console script: `hello-science ...`

`python -m` is underrated

- Runs a module as a script
- Works well with packages

```
python -m hello_science.cli --name Ada
```

Add a console script entry point

In `pyproject.toml`:

```
[project.scripts]
hello-science = "hello_science.cli:main"
```

Then:

```
pip install -e .
hello-science --name Ada
```

Why entry points?

- Users don’t need to know file locations
- PATH-based execution
- Standard for tools (pytest, black, pip, …)

Lightweight task runners (optional)

- `make`, `just`, `nox`, `tox`
- Example `make test`, `make format`

(Show later in “Quality tools” / “Testing” sections.)

8) Configuration management

CLI args vs config files

- CLI args: great for small parameter sets
- Config files: better for many parameters / experiments

Often: CLI selects config file.

Example config (TOML)

`config.toml`:

```toml
name = "Ada"
seed = 123
output = "out.txt"
```

Reading config in Python

```python
from pathlib import Path
import tomllib  # Python 3.11+

def load_config(path: Path) -> dict:
    with path.open("rb") as f:
        return tomllib.load(f)
```

Merging config + CLI overrides

Pattern:

- load config defaults
- apply CLI overrides when provided

Keep precedence rules simple and documented!

Environment variables

Useful for:

- secret tokens (not in Git)
- machine-specific paths

```
import os
cache_dir = os.environ.get("HELLO_CACHE", ".cache")
```

9) Logging instead of printing

Why logging?

- Control verbosity (`INFO`, `DEBUG`, `WARNING`)
- Route messages to file for long runs
- Cleaner than scattered `print()`

Minimal logging setup

```python
import logging

logging.basicConfig(level=logging.INFO)
log = logging.getLogger("hello_science")

log.info("Starting run")
```

Logging in library code

In `core.py`:

```python
import logging
log = logging.getLogger(__name__)

def greet(name: str) -> str:
    log.debug("Generating greeting for %s", name)
    return f"Hello, {name}!"
```

CLI controls verbosity

```
p.add_argument("-v", "--verbose", action="count", default=0)
```

Map to logging levels:

- 0 → WARNING
- 1 → INFO
- 2+ → DEBUG

Logging best practices

- never log secrets
- avoid huge logs in tight loops
- include context: parameters, input file names, versions

10) File I/O and paths

Use `pathlib`

```python
from pathlib import Path

out = Path("results") / "hello.txt"
out.parent.mkdir(parents=True, exist_ok=True)
out.write_text("Hello!\n", encoding="utf-8")
```

Read and write structured data

Examples (depending on course focus):

- CSV: `pandas.read_csv`, `DataFrame.to_csv`
- JSON: `json.load/dump`
- NumPy arrays: `np.save`, `np.load`

Add output to hello example

```python
def write_greeting(path: Path, text: str) -> None:
    path.write_text(text + "\n", encoding="utf-8")
```

CLI:

```bash
hello-science --name Ada --out out.txt
```

Handle encodings explicitly

- Especially important for text processing (nltk)

```
Path("in.txt").read_text(encoding="utf-8")
```

Robust file behavior

- Validate inputs exist
- Avoid overwriting by default (or add `--force`)
- Create directories as needed

11) Testing with pytest

Why tests in scientific code?

- Prevent regressions when refactoring notebooks → modules
- Validate known cases
- Make results reproducible across machines

First test: pure function

tests/test_core.py

```
from hello_science.core import greet

def test_greet():
    assert greet("Ada") == "Hello, Ada!"
```

Run:

```
pytest
```

Parametrized tests

```python
import pytest
from hello_science.core import greet

@pytest.mark.parametrize("name", ["world", "Ada", ""])
def test_greet_names(name):
    assert greet(name).startswith("Hello, ")
```

- runs the test once with each of the paramters

Testing I/O (temporary paths)

`tmp_path` is a built-in **pytest fixture**:

- pytest creates a fresh **temporary directory** for this test (as a `pathlib.Path`)
- it is passed in via **fixture injection by name** (parameter `tmp_path`)
- temp dir is unique per test and cleaned up after the run

```
1 from pathlib import Path
2 from hello_science.io import write_greeting
3
4 def test_write_greeting(tmp_path: Path):  # pytest provides this argument
5     p = tmp_path / "out.txt"
6     write_greeting(p, "Hello, Ada!")
7     assert p.read_text(encoding="utf-8").strip() == "Hello, Ada!"
```

Testing guidance

- prioritize pure functions
- keep tests fast
- if randomness: fix seeds; test distributions with tolerances

12) Style and quality tools

Why bother?

- Consistent code is easier to read and grade
- Automated checks reduce human review effort
- Industry-standard workflows

Formatter: Black (example)

```
pip install black
black src tests
```

Outcome:

- standardized formatting, no bikeshedding

Linting: Ruff (example)

```
pip install ruff
ruff check src tests
```

Finds:

- unused imports
- common bugs
- style issues

Pre-commit hooks (optional but powerful)

`.pre-commit-config.yaml` (conceptual)

- run black/ruff before every commit

```
pre-commit install
git commit ...
```

Minimal “quality” Makefile (optional)

```makefile
format:
    black src tests

lint:
    ruff check src tests

test:
    pytest -q
```

13) Type hints and documentation

Type hints: what they give you

- better IDE autocomplete
- catch some bugs earlier
- self-documenting function signatures

Add hints to core

```python
def greet(name: str) -> str:
    return f"Hello, {name}!"
```

More realistic:

```python
from pathlib import Path

def write_greeting(path: Path, text: str) -> None:
    path.write_text(text + "\n", encoding="utf-8")
```

Static checking (optional)

```
pip install mypy
mypy src
```

- mypy is a tool, not a religion
- use it where it helps (APIs, refactoring)

Docstrings (NumPy style example)

```python
def greet(name: str) -> str:
    """
    Create a greeting string.

    Parameters
    ----------
    name:
        Name to greet.

    Returns
    -------
    str
        Greeting text.
    """
    return f"Hello, {name}!"
```

Docs build (overview)

- Sphinx / MkDocs can render docstrings + pages
- Even without full docs:
  - good README + `--help` + docstrings go far

14) Debugging outside notebooks

Read tracebacks effectively

- find the *first* line in your code
- identify the exception type (`ValueError`, `FileNotFoundError`, ...)

`breakpoint()` / pdb

`breakpoint()` starts the Python debugger (pdb) **at that line**. Execution pauses, and you can inspect variables step-by-step in the terminal.

```
def greet(name: str) -> str:
    breakpoint()              # pause here and open pdb
    return f"Hello, {name}!"
```

Run your program normally:

```
python -m hello_science.cli --name Ada
```

`breakpoint()` / pdb

What happens:

- Python stops at `breakpoint()` and shows a `(Pdb)` prompt
- you can inspect state (`p name`), step (`n`), continue (`c`), quit (`q`)

```
(Pdb) p name
'Ada'
(Pdb) n
(Pdb) c
```

Logging for debugging

Instead of prints:

```
log.debug("args=%r", args)
```

Enable with `--verbose`.

IDE debugging

- VS Code / PyCharm:
  - run configurations
  - breakpoints
  - variable inspector
- Works better with scripts than notebooks for many workflows

Minimal reproducible example

- Reduce a bug to:
  - smallest dataset
  - smallest script
- Helps asking questions and writing bug reports

15) Performance and profiling (lightweight)

Don’t guess: measure

- Use timing tools rather than intuition

```
python -m timeit -n 1000 "sum(range(1000))"
```

Quick timing in code

```python
import time

t0 = time.perf_counter()
# do work
dt = time.perf_counter() - t0
print(f"took {dt:.3f}s")
```

Profiling with `cProfile`

A profiler measures **where your program spends time** (which functions are slow). `cProfile` records function call counts and cumulative time and prints a report.

Profile the CLI run:

```
python -m cProfile -s cumulative -m hello_science.cli --name Ada
```

What happens:

- `-m hello_science.cli` runs your module like a script
- `-m cProfile` wraps the whole run and records timings
- `-s cumulative` sorts by “time spent in this function + subcalls”

Profiling with `cProfile`

How to read the table (key columns):

- `ncalls`: how often a function was called
- `tottime`: time *in the function body* (excluding subcalls)
- `cumtime`: time in the function *including* subcalls (often most useful)

Use it to identify hotspots → then optimize the *few* functions that dominate runtime.

Common performance wins in scientific Python

- vectorize with NumPy
- avoid Python loops in hotspots
- avoid repeated file I/O
- cache intermediate results when appropriate

Performance advice

- optimize only after correctness + tests
- keep copy-pasteable benchmark command (or two) in `README.md`
  - **Reproducible performance checks:** you (or others) can rerun the *same* benchmark after code changes and compare.
  - **Avoid accidental slowdowns:** simple “before/after” timing helps catch regressions during refactors.
  - **Clear expectations:** documents what “typical workload” and “typical runtime” mean for the project.
  - **Better bug reports/discussion:** others can report “this command is slow on my machine” with a shared reference.

16) Reproducible workflows

What “reproducible run” means

Given:

- same code version
- same dependencies
- same inputs
- same parameters

→ get same outputs (or acceptably close)

Record metadata

- versions: Python, numpy, sklearn
- command line invocation
- git commit hash (optional but useful)

Example:

```
import sys, platform
import numpy as np
print(sys.version)
print(platform.platform())
print(np.__version__)
```

Randomness: seed + document

- always pass a seed parameter
- include it in output filenames or logs

Save outputs deterministically

- Avoid ambiguous filenames like `results.csv` that get overwritten.
- Prefer names derived from parameters (and/or a timestamp).

Save outputs deterministically

Example: include key parameters in the output name

```python
from pathlib import Path

def output_path(base_dir: Path, seed: int, n: int) -> Path:
    run_id = f"seed{seed}_n{n}"
    return base_dir / run_id / "greeting.txt"

out = output_path(Path("runs"), seed=123, n=1000)
out.parent.mkdir(parents=True, exist_ok=True)
out.write_text("Hello, Ada!\n", encoding="utf-8")
print("Wrote", out)
```

Creates e.g. `runs/seed123_n1000/greeting.txt

Organize experiments

Simple pattern:

```
runs/
  2026-01-19_seed123/
    config.toml
    out.txt
    log.txt
```

17) Version control integration

Why Git matters here

- scripts/packages yield clean diffs
- tests and CI can run automatically
- easier collaboration and grading

`.gitignore` essentials

Typical ignores:

- `.venv/`
- `__pycache__/`
- `.ipynb_checkpoints/`
- `*.pyc`
- `dist/`, `build/`
- large outputs: `results/`, `runs/`

Commit structure

- small commits with messages:
  - “Add CLI parsing”
  - “Refactor core into package”
  - “Add tests for greet()”
- makes it easier to review and revert

Notebooks in Git: pragmatic approach

- keep “final” notebooks without output (or with minimal output)
- rely on scripts for official pipeline
- optionally use tools like `nbstripout` (concept)

Tagging releases

For assignments/papers:

- create a git tag or release
- ties code to a submission version

18) Mini capstone: refactor notebook → package

Starting point

- A notebook does:
  - load data
  - compute something
  - save output

Goal:

- move compute into `core.py`
- move I/O + args into `cli.py`

Target CLI behavior

```
hello-science --name Ada --out out.txt --seed 123 -v
```

Outputs:

- greeting written to `out.txt`
- logs show parameters

Suggested module split

- `core.py`: `greet()`, `simulate()` (pure)
- `io.py`: `write_greeting()`
- `cli.py`: parse args, setup logging, call core+io

Add tests + quality gates

- `pytest` for correctness
- `ruff`/`black` for style
- (optional) GitHub/GitLab CI to run on every push

Deliverable

A repository that:

- installs with `pip install -e .`
- runs from terminal (`hello-science --help`)
- has `pytest` passing
- includes README with:
  - install instructions
  - one example command

Appendix: consolidated code (reference)

src/hello_science/cli.py

```python
import argparse
import logging
from pathlib import Path

from .core import greet
from .io import write_greeting

def parse_args(argv=None):
    p = argparse.ArgumentParser(prog="hello-science")
    p.add_argument("--name", default="world")
    p.add_argument("--out", type=Path, default=Path("hello.txt"))
    p.add_argument("--seed", type=int, default=0)
    p.add_argument("-v", "--verbose", action="count", default=0)
    return p.parse_args(argv)

def setup_logging(verbose: int) -> None:
    level = logging.WARNING
    if verbose == 1:
```

src/hello_science/core.py

```python
import logging
log = logging.getLogger(__name__)

def greet(name: str) -> str:
    log.debug("Generating greeting for name=%r", name)
    return f"Hello, {name}!"
```

src/hello_science/io.py

```python
from pathlib import Path

def write_greeting(path: Path, text: str) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(text + "\n", encoding="utf-8")
```

tests/test_core.py

```python
from hello_science.core import greet

def test_greet():
    assert greet("Ada") == "Hello, Ada!"
```

pyproject.toml (sketch)

```toml
[project]
name = "hello-science"
version = "0.1.0"
dependencies = []

[project.scripts]
hello-science = "hello_science.cli:main"

[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[tool.setuptools.packages.find]
where = ["src"]
```

Questions?

End

of Session 13